一、ajax_分页

二、教学目标

1、掌握ajax核心能力

2、理解分页概念

3、准备初始数据完成手动分页

三、ajax 的回调函数

- 后端使用 HttpResponse 返回响应数据是以字符串的形式返回,回调函数res 接收到的就是字符串,不能 res.属性获取值
- 后端使用 JsonResponse 返回字典格式的数据回调函数会自动序列化,将 python 字典格式转成前端的自定义对象,所以在回调函数中就可以通过 res.属性获取值

```
def ajax_test(reqeust):
    if reqeust.method == 'POST':
         d1 = request.POST.get('d1')
        d2 = request.POST.get('d2')
        # #相加
        sum = int(d1) + int(d2)
        # #返回结果
        # return HttpResponse(sum)

        #返回字典
        back_dict = {'code':200,'msg':'响应成功','data':sum}
        #实际上返回的就是json数据
        return JsonResponse(back_dict)


    return render(reqeust,'ajaxtest.html')
```

ajaxtest.html

```
//4. success: 成功回调函数 有结果时自动执行
           success:function(res){
                //有结果会自动执行
                 //res 是接收后端返回结果的数据
                 //alert(res)
                 //console.log(res.code)
                 //console.log(res.msg)
                 if(res.code ==200){
                     $('#sum').html(res.data)
                 }

           }
```

四、前后端传输数据的编码格式(contentTypes)

前后端进行数据传输的编码格式

- urlencoded
- multipart/form-data
- json

后端提交 post 请求的方式

4.0.1 form表单

- 默认编码格式是 urlencode数据格式是:username=xxx&password=xxx
- django 会自动将 urlencoded 编码格式的数据封装到 request.POST中
- 编码格式变成formdata之后,django会自动将普通键值对封装在request.POST中,将文件封装在request.FILES中

  注意: form 表单不能提交 json 格式数据

urls.py

```
 path('index/',views.index)
```

app01.py -> views.py

```
def index(request):
    if request.method =='POST':
        #获取到post请求的数据 编码格式变成formdata之后,django会自动将普通键值对封装在
request.POST中
        print(request.POST)
        #POST请求 将文件封装在request.FILES中
        print(request.FILES)
    return render(request,'index.html')
```

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>
    {% load static %}

    <!-- 导入外部的css-->
    <link rel="stylesheet" href={% static 'bootstrap-3.4.1-
dist/css/bootstrap.min.css' %}>
    <!-- 导入外部的js-->
    <script src={% static "js/jquery-3.7.0.min.js" %}></script>
    <!-- 导入外部的js-->
    <script src={% static "bootstrap-3.4.1-dist/js/bootstrap.min.js" %}>
</script>

</head>
<body>
<form action="" method="post" enctype="multipart/form-data">
    <p>用户名: <input type="text" name="username"> </p>
    <p>密码: <input type="password" name="password"> </p>
    <p>文件: <input type="file" name="file"></p>
    <input type="submit">
</form>
</body>
</html>
```

4.0.1 ajax请求

- 默认编码格式是 urlencode数据格式是:username=xxx&password=xxx
- django 会自动将 urlencoded 编码格式的数据封装到 request.POST中
- 发送JSON格式的数据
  - 将 contentType 改成 application/json
  - 后端使用request.body 来接收数据
- 发送文件数据
  - 需要借助内置对象FormData
    - 添加普通的键值对数据
    - $('#file')[0].files[0] 获取文件的值
  - 需要指定两个重要参数
    - contentType:false,//不要任何编码,django能自动识别FormData对象
    - processData:false,//让浏览器不要对数据进行处理

注意:django 后端不会自动处理 json 格式的数据,需要我们自己手动对request.body进行反序列化

urls.py

```
path('ajax/', views.ajax)
```

app01.py -> views.py

```
def ajax(request):
    if request.method =='POST':
        #这样是获取不到JSON数据
        #print(request.POST)
        #print(request.body) #二进制数据
        #需要将二进制数据进行反序列化，转成字典
        #dict=json.loads(request.body)
        #print(dict)
        #print(dict['username'])
        #print(dict['age'])

        #django 后端能直接识别 FormData 对象并将普通键值对数据封装在request.PosT中，
        #将文件数据封装在 request.FILES 中
        print(request.POST)
        print(request.FILES)

    return render(request,'ajax.html')
```

ajax.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>
    {% load static %}

    <!-- 导入外部的css-->
    <link rel="stylesheet" href={% static 'bootstrap-3.4.1-
dist/css/bootstrap.min.css' %}>
    <!-- 导入外部的js-->
    <script src={% static "js/jquery-3.7.0.min.js" %}></script>
    <!-- 导入外部的js-->
    <script src={% static "bootstrap-3.4.1-dist/js/bootstrap.min.js" %}>
</script>

</head>
<body>
<input type="button" class="btn btn-success" id="d1" value="文本">
<input type="button" class="btn btn-success" id="d2" value="JSON">
<p>
     <p>用户名: <input type="text" name="username" id="username"> </p>
    <p>密码: <input type="password" name="password" id="password"> </p>
    <p>文件: <input type="file" name="file" id="file"></p>
    <input type="button"class="btn btn-primary" id="d3" value="文件">
</p>


</body>

<script>

    ....
     //发送普通文本以及文件数据
    $('#d3').click(function () {
```

```
        //初始化数据对象
        let formDataObj = new FormData()
        //获取普通文本
        var username = $('#username').val()
        var password = $('#password').val()
        //获取文件对象数据
        //$('#file') 获取file对象
        //$('#file')[0] 获取input标签对象本身
        //$('#file')[0].files 获取input标签中上传的文件数据,返回文件列表
        //$$('#file')[0].files[0] 获取文件列表中的文件对象
        var file=$('#file')[0].files[0]

        //键值对
        formDataObj.append('username',username)
        formDataObj.append('password',password)
        formDataObj.append('file',file)

        $.ajax({
            url:'',
            type:'post',
            data:formDataObj,
            contentType:false,//不要任何编码,django能自动识别FormData对象
            processData:false,//让浏览器不要对数据进行处理
            success:function(res){


            }

        });
    })

</script>
</html>
```

五、分页

5.0.1 准备数据

直接使用原生的数据库

models.py

```
from django.db import models

# Create your models here.
class Book(models.Model):

    title = models.CharField(max_length=32)
```

迁移数据库

```
python manage.py makemigrations
```

执行数据库

```
python manage.py migrate
```

urls.py

```
path('book/',views.book)
```

views.py

```
from app01 import models

def book(request):
    #往book表中插入1000条数据
    for i in range(1000):
        models.Book.objects.create(title='第%s本书' % (i+1))
    #查询数据
    book_queryset = models.Book.objects.all()
    #返回
    return render(request,'book.html',locals())
```

book.html

```
<body>
{% for book_obj in book_queryset %}<p>
    {{ book_obj.title }}</p>
{% endfor %}
</body>
```

这样发现问题 太慢了 因为是持续的加载数据和查询数据

改进

views.py

```
def book(request):
    #往book表中插入1000条数据
    #for i in range(1000):
    #    models.Book.objects.create(title='第%s本书' % (i+1))
    #查询数据
    #book_queryset = models.Book.objects.all()

    # 批重操作
    book_list = []
    for i in range(10000):
        book_obj = models.Book(title='第%s本书' % (i + 1))  # 通过类实例化对象
        book_list.append(book_obj)#批添加数据
    models.Book.objects.bulk_create(book_list)
    book_queryset =models.Book.objects.all()

    #返回
    return render(request,'book.html',locals())
```

5.0.1 分页

页面模版

```
<nav>
  <ul class="pagination">
    <li class="page-item">
      <a class="page-link" href="#" aria-label="Previous">
        <span aria-hidden="true">&laquo;</span>
      </a>
    </li>
    <li class="page-item"><a class="page-link" href="#">1</a></li>
    <li class="page-item"><a class="page-link" href="#">2</a></li>
    <li class="page-item"><a class="page-link" href="#">3</a></li>
    <li class="page-item"><a class="page-link" href="#">4</a></li>
    <li class="page-item">
      <a class="page-link" href="#" aria-label="Next">
        <span aria-hidden="true">&raquo;</span>
      </a>
    </li>
  </ul>
</nav>
```

urls.py

```
path('fenye/',views.fenye)
```

app01 -> views.py

```
def fenye(request):

    book_queryset = models.Book.objects.all()

    # 获取当前页的数据
    page_num = int(request.GET.get('page',1))#默认显示第一页
    # 一页展示多少条数据
    per_page = 50
    # 每页的起始位置和结束位置
    #起始位置 = (当前页 - 1) × 每页数量
    #结束位置 = 当前页 × 每页数量
    start_num = (page_num-1) * per_page
    end_num = page_num * per_page
    #总页数
    total_num = book_queryset.count() #总条数
    # 总页数 = (总数据量 + 每页数量 - 1) // 每页数量（向上取整算法）
    #200条记录/每页50条 → (200+50-1)//50 = 249//50 = 4页
    total_page = (total_num + per_page -1) // per_page #总页数

    page_html = ''
    for i in range(1,total_page+1):
        page_html += '<li class="page-item"><a class="page-link" href="?
page=%s">%s</a></li>' %(i,i)

    #切片获取当前页的数据
```

```
    # 获取当前页的数据
    # 使用切片操作获取当前页的数据范围
    # Django ORM的切片操作会高效地转化为SQL的LIMIT和OFFSET语句
    book_queryset = book_queryset[start_num:end_num]

    # 返回
    return render(request, 'fenye.html', locals())
```

fenye.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>


    {% load static %}

    <!-- 导入外部的css-->
    <link rel="stylesheet" href={% static 'bootstrap-3.4.1-
dist/css/bootstrap.min.css' %}>
    <!-- 导入外部的js-->
    <script src={% static "js/jquery-3.7.0.min.js" %}></script>
    <!-- 导入外部的js-->
    <script src={% static "bootstrap-3.4.1-dist/js/bootstrap.min.js" %}>
</script>

</head>
<body>
{% for book_obj in book_queryset %}<p>
    {{ book_obj.title }}</p>
{% endfor %}
<nav>
  <ul class="pagination">
     <!-- 上一页按钮 -->
    <li class="page-item {% if page_num == 1 %}disabled{% endif %}">
      <a class="page-link" href="?page={% if page_num > 1 %}{{ page_num|add:-1
}}{% else %}1{% endif %}" aria-label="Previous">
        <span aria-hidden="true">&laquo;</span>
      </a>
    </li>

    <!-- 中间页码部分 -->
    {{ page_html|safe }}

    <!-- 下一页按钮 -->
    <li class="page-item {% if page_num == total_page %}disabled{% endif %}">
      <a class="page-link" href="?page={% if page_num < total_page %}{{
page_num|add:1 }}{% else %}{{ total_page }}{% endif %}" aria-label="Next">
        <span aria-hidden="true">&raquo;</span>
      </a>
    </li>
  </ul>
</nav>
```

```
</body>
</html>
```

六、作业

6.1 完成本天的案例